色块识别-灰度图

色块识别-灰度图

- 例程效果
- 代码讲解
  - 代码片段讲解
  - 导入模块
    - 设置图像尺寸
    - 初始化摄像头
    - 初始化显示模块
  - 3.5 初始化媒体管理器并启动摄像头
    - 设置连通域检测参数
    - 图像处理与 Blob 检测
    - 资源释放
  - 参数调整说明

例程效果

运行本节例程代码 [源码汇总/17.cv_lite/1.grayscale_find_blobs.py]

本节是基于灰度图像的色块识别，K230会检测画面中阈值符合设定的色块，并用矩形框标记出来

> 观察串行终端输出的帧率信息，在供电稳定的情况下帧率可以稳定在80帧以上



代码讲解

代码片段讲解

导入模块

```
import time, os, sys, gc
from machine import Pin
from media.sensor import *    # 摄像头模块
from media.display import *   # 显示模块
from media.media import *     # 媒体管理器
import _thread
import cv_lite                # cv_lite 扩展模块
import ulab.numpy as np       # 轻量 NumPy
```

这些模块提供了摄像头控制、图像显示、内存管理和 Blob 检测所需的功能。

设置图像尺寸

```
image_shape = [480, 640]  # 高 x 宽
```

定义图像的分辨率为 480x640（高度 x 宽度），后续会根据此尺寸初始化摄像头和显示模块。

初始化摄像头

```
sensor = Sensor(id=2, width=1280, height=720, fps=90)  # 构建 Sensor 对象
sensor.reset()                                         # 复位摄像头
sensor.set_framesize(width=image_shape[1], height=image_shape[0])  # 设置帧大小
sensor.set_pixformat(Sensor.GRAYSCALE)                 # 设置为灰度图输出
```

- 初始化摄像头，设置分辨率为 1280x720，帧率为 90 FPS。
- 将输出帧大小调整为 640x480，并设置为灰度图模式（单通道图像，节省内存和计算资源）。

初始化显示模块

```
Display.init(Display.ST7701, width=image_shape[1], height=image_shape[0],
             to_ide=True, quality=50)
```

初始化显示模块，使用 ST7701 驱动芯片（即K230豪华版的LCD屏幕），分辨率与图像尺寸一致。
`to_ide=True` 表示图像会同时传输到 IDE 进行虚拟显示，`quality=50` 设置图像传输质量。

3.5 初始化媒体管理器并启动摄像头

```
MediaManager.init()
sensor.run()  # 启动摄像头图像流
```

初始化媒体资源管理器，并启动摄像头开始捕获图像流。

设置连通域检测参数

```
threshold = [230, 255]   # 二值化阈值范围（亮区域）
min_area = 10            # 最小目标面积
kernel_size = 1          # 腐蚀核大小（用于降噪）
clock = time.clock()     # 帧率计时器
```

- `threshold`：二值化阈值范围，灰度值在 230 到 255 之间的像素被认为是目标区域（适用于检测亮区域）。
- `min_area`：连通域的最小面积，小于此值的区域会被忽略。
- `kernel_size`：图像腐蚀操作的核大小，用于去除噪声。
- `clock`：用于计算帧率。

图像处理与 Blob 检测

```
while True:
    clock.tick()

    # 获取一帧图像
    img = sensor.snapshot()
    img_np = img.to_numpy_ref()  # 获取图像数据引用

    # 执行灰度图二值连通域检测
    blob = cv_lite.grayscale_find_blobs(
        image_shape, img_np,
        threshold[0], threshold[1],
        min_area, kernel_size
    )

    # 若检测到连通区域
    if len(blob) > 0:
        # blob = [x, y, w, h]
        img.draw_rectangle(blob[0], blob[1], blob[2], blob[3],
                           color=(0, 0, 0), thickness=2)  # 绘制黑色矩形

    # 显示图像
    Display.show_image(img)

    # 清理内存并打印帧率
    gc.collect()
    print("findblobs:", clock.fps())
```

- **图像捕获**：通过 `sensor.snapshot()` 获取一帧图像，并通过 `to_numpy_ref()` 将其转换为 NumPy 数组引用。
- **Blob 检测**：调用 `cv_lite.grayscale_find_blobs()` 函数进行连通域检测，返回检测到的区域信息（格式为 `[x, y, w, h]`，即左上角坐标和宽度、高度）。
- **绘制矩形**：如果检测到 Blob，则在图像上绘制一个黑色矩形框标注目标区域。
- **显示图像**：将处理后的图像显示到屏幕或 IDE 虚拟窗口。
- **内存管理和帧率输出**：调用 `gc.collect()` 清理内存，并通过 `clock.fps()` 打印当前帧率。

资源释放

```
sensor.stop()
Display.deinit()
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
time.sleep_ms(100)
MediaManager.deinit()
```

程序退出时，停止摄像头、释放显示模块资源，并清理媒体管理器。

参数调整说明

- `threshold`：根据环境光线调整二值化阈值范围。如果目标区域较暗，可降低范围（如 `[100, 200]`）；如果目标区域较亮，可提高范围（如 `[200, 255]`）。
- `min_area`：根据目标大小调整此值，较小的值会检测到更多小区域，较大的值会过滤掉噪声。
- `kernel_size`：用于图像腐蚀，值越大，降噪效果越强，但可能丢失小目标的细节。